Jumătatea a 2-a a semestrului

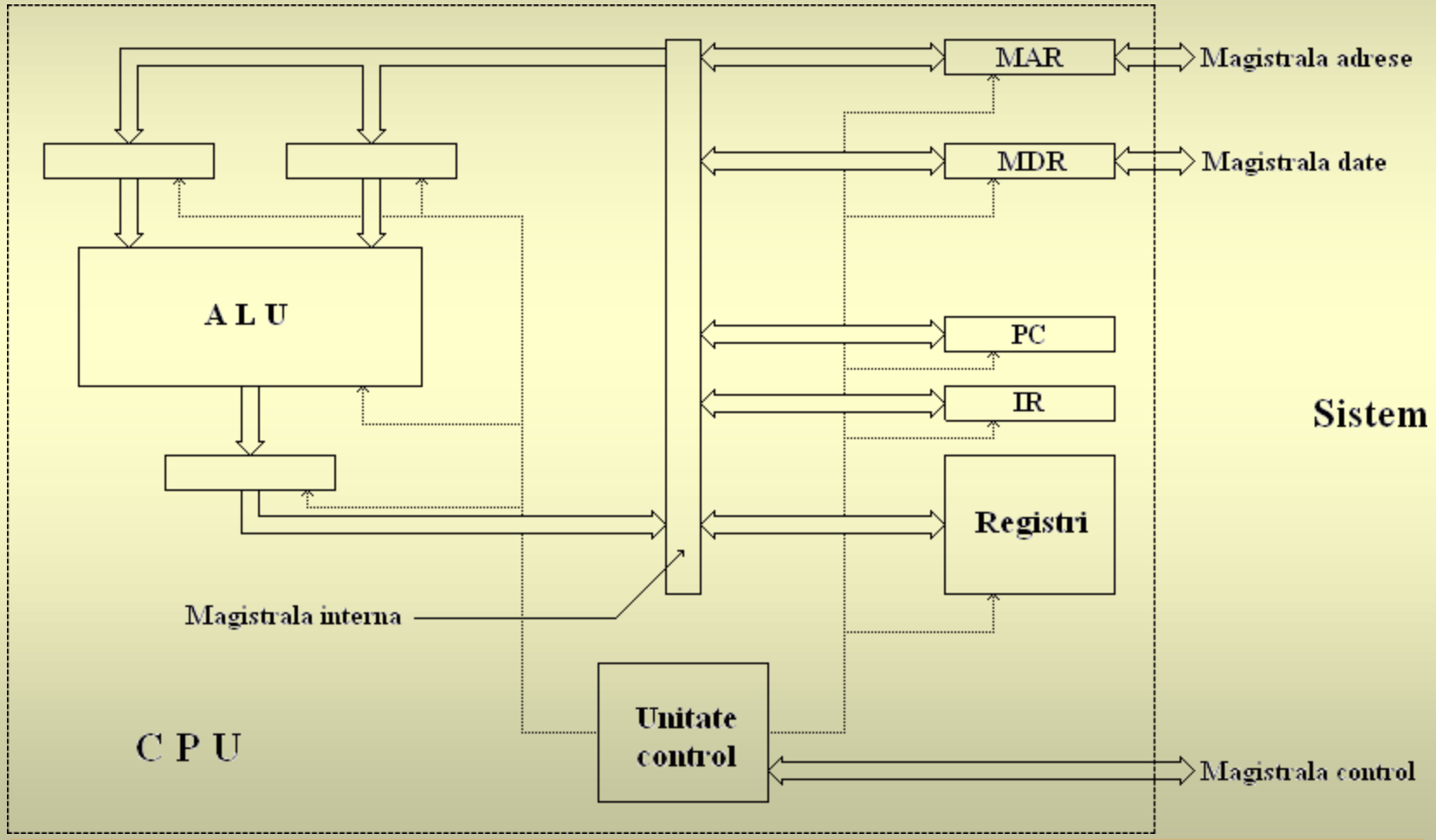
- curs
  - structura hardware a unui sistem de calcul
  - limbajul de asamblare
- laborator
  - limbajul de asamblare Intel
  - finalizare - test practic
    - condiţie susţinere test - cel mult 2 absenţe la laborator

Cuprins - structura hardware

I. Unitatea centrală de procesare (CPU)
II. Dispozitivele periferice
III. Sistemul de întreruperi
IV. Sistemul de operare

I. Unitatea centrală de procesare (CPU)

Structura CPU (1)



Structura CPU (2)

- unitatea aritmetică şi logică (ALU)
  - efectuează calculele propriu-zise
- regiştrii de uz general
- unitatea de control
  - comandă celelalte componente
  - stabileşte ordonarea temporală a operaţiilor
- magistrala internă

Structura CPU (3)

- contorul program (PC)
  - reţine adresa următoarei instrucţiuni de executat
  - actualizat de procesor
  - uzual nu este accesibil prin program
- registrul de instrucţiuni (IR)
  - reţine codul ultimei instrucţiuni aduse din memorie

Structura CPU (4)

- regiştrii de interfaţă
  - asigură comunicarea cu magistralele sistemului
  - de adrese: MAR (*Memory Address Register*)
  - de date: MDR (*Memory Data Register*)
- regiştrii temporari
  - intermediari între diverse componente
  - exemple: regiştrii operanzi ALU, registrul rezultat ALU

Cum putem creşte performanţa?

- eliminarea factorilor care frânează CPU
  - exemplu - folosirea memoriei cache
- structuri cât mai simple
  - nu mai este posibil la procesoarele actuale
- creşterea frecvenţei ceasului
  - limitată de tehnologie
- execuţia instrucţiunilor în paralel

I.1. Pipeline

Ideea de pornire

- execuţia unei instrucţiuni - număr mare de paşi
- în paşi diferiţi se folosesc resurse diferite ale CPU
- execuţia unei instrucţiuni poate începe înainte de terminarea celei anterioare
- instrucţiunile se execută (parţial) în paralel

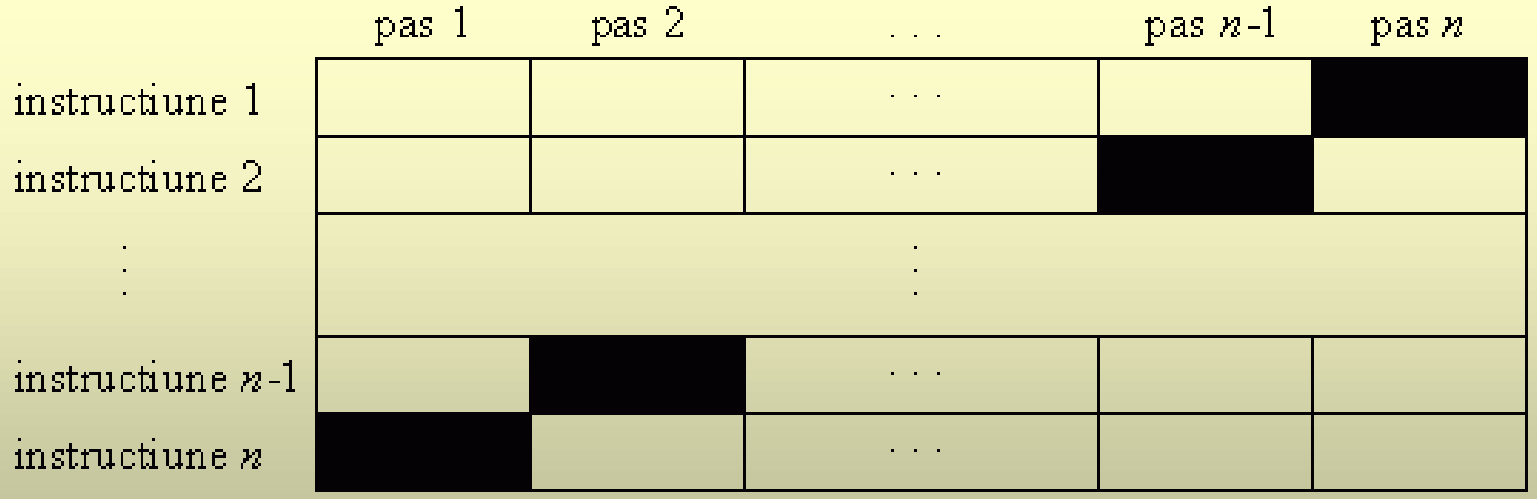
O primă implementare

Procesorul Intel 8086

- format din două unităţi
  - unitatea de interfaţă cu magistrala (BIU)
    - comunicarea cu exteriorul
  - unitatea de execuţie (EU)
    - execuţia propriu-zisă a operaţiilor
- BIU şi EU pot lucra în paralel

Principiul benzii de asamblare

- execuţia unei instrucţiuni - *n* paşi
- la un moment dat - *n* instrucţiuni în execuţie
- fiecare instrucţiune - în alt pas



Pipeline

- secvenţa paşilor (*stagii*) prin care trece execuţia unei instrucţiuni
- trecerea între două stagii - la fiecare ciclu de ceas
- cât durează până la terminarea unei instrucţiuni?
  - prima instrucţiune - *n* cicluri de ceas
  - următoarele instrucţiuni - câte 1 ciclu de ceas !

Performanţa unui pipeline

- rezultatul obţinut la fiecare stagiu trebuie reţinut
- regiştri de separaţie - plasaţi între stagii
- frecvenţa ceasului - dată de stagiul cel mai lung
- paşi mai simpli
  - număr de stagii mai mare
  - frecvenţă mai mare a ceasului

Execuţia unei instrucţiuni

1. depunerea valorii PC (adresa instrucţiunii) în MAR
2. citirea din memorie
3. preluarea codului instrucţiunii în MDR
4. depunerea codului instrucţiunii în IR
5. actualizarea valorii PC

Execuţia unei instrucţiuni

6. decodificarea instrucţiunii de către unitatea de control
7. citire operand din memorie
   - depunere adresă operand în MAR
   - comandă citire
   - preluare operand în MDR

7'. selecţie registru care conţine operandul

Execuţia unei instrucţiuni

8. depunere operand în registru operand ALU
9. repetare paşi 7-8 pentru al doilea operand
10. transmiterea către ALU a codului operaţiei dorite
11. preluare rezultat în registrul rezultat ALU
12. testare condiţie salt

Execuţia unei instrucţiuni

13. salt (dacă este cazul)

14. scriere rezultat în memorie

- depunere rezultat în MDR
- depunere adresă în MAR
- comandă scriere

14'. scriere rezultat în registrul destinaţie

Probleme

- nu toate instrucţiunile se pot executa în paralel
- dependenţă - o instrucţiune trebuie să aştepte terminarea alteia
- conflict în accesul la aceeaşi resursă

Tipuri de dependenţe

- structurale
- de date
- de control

Dependenţe structurale

- instrucţiuni aflate în stagii diferite au nevoie de aceeaşi componentă
- o singură instrucţiune poate folosi componenta la un moment dat
- celelate instrucţiuni care au nevoie de ea sunt blocate

Dependenţe structurale - exemple

- ALU
  - instrucţiuni aritmetice
  - calculul adreselor operanzilor
  - actualizarea valorii PC
- accesele la memorie
  - citire cod instrucţiune
  - citire operand
  - scriere rezultat

Dependenţe de date

- o instrucţiune calculează un rezultat, alta îl foloseşte
- a doua instrucţiune are nevoie de rezultat înainte ca prima să-l obţină
- a doua instrucţiune este blocată

Dependenţe de date - exemplu

```
mov eax,7
sub eax,3
```

- prima instrucţiune: scrierea în `eax` - în ultimul stagiu
- a doua instrucţiune: utilizarea `eax` - în primele stagii (decodificare)
  - aşteaptă până când prima instrucţiune depune rezultatul în `eax`

Dependenţe de control (1)

Actualizarea valorii PC (uzual)

- adunarea la vechea valoare a dimensiunii codului instrucţiunii anterioare
- încărcarea unei valori noi - instrucţiuni de salt

Dependenţe de control (2)

Tipuri de instrucţiuni de salt

- necondiţionat
  - se face saltul întotdeauna
- condiţionat
  - se face saltul numai dacă este îndeplinită o anumită condiţie
  - altfel se continuă cu instrucţiunea următoare

Dependenţe de control (3)

Adresa de salt - moduri de exprimare

- valoare constantă
  - absolută
  - deplasament faţă de adresa instrucţiunii curente
- valoarea dintr-un registru
- valoarea dintr-o locaţie de memorie

Dependenţe de control (4)

Probleme

- calculul adresei de salt - în ultimele stagii de execuţie
- instrucţiunile următoare (multe!) au început deja execuţia
- dacă se face salt - efectele lor trebuie anulate

Dependenţe de control (5)

Probleme

- "golirea" pipeline-ului → pierdere de performanţă
  - operaţii complicate
  - durează mult până la terminarea primei instrucţiuni → scade rata de execuţie
- o instrucţiune din 7 (în medie) este de salt !